

Datapath Architecture

Implementation

Implemented entirely in `design/core.cc`, one `Core` method per stage, evaluated once per cycle in fixed reverse order: **Writeback** → **Memory** → **Execute** → **Decode** → **Fetch**. Briefly: **Fetch** delivers a fetched instruction into the real `if_id_latch` and issues the next request; **Decode** runs five ordered checks then dispatches by calling `Operate()` directly on the chosen pipeline (no shared ID/EX latch); **Memory** harvests completed results and hands AGU's address to the cache; **Writeback** retires the oldest completed instruction, the only point architectural state changes.

Execute and the cache connections, in detail. Each of the seven units—ADD, SUB, AND, OR, XOR, CONTROL, AGU—is a stateful pipeline with configurable latency and II; nominal result boundary is `dispatch + latency + 1`. Decode dispatches to exactly one of these seven every cycle via `kClockedData`. Six of them—ADD, SUB, AND, OR, XOR, CONTROL—feed their result back into Execute (`kData`) once their latency elapses; Execute is simply what advances all seven pipelines one internal stage per cycle, skipping whichever one Decode just dispatched to. **AGU is the exception:** it does not feed Execute at all, it feeds **Memory** directly, because a load/store's job is to produce a cache address, not an ALU-style result—this is also what the validator itself requires (*“AGU lacks its real result path”* unless wired to Memory).

From there, Memory is the only stage that ever touches the cache. It calls `TryIssueData` to hand AGU's computed address to **L1D**, which feeds **L2D** in turn—this is the *only* wired path from AGU to the data cache; AGU never talks to L1D directly. On the instruction side, Fetch's requests go to **L1I**, which feeds **L2I**. The two hierarchies are completely independent controllers with no shared state between them; L1I/L2I never see a data address, and L1D/L2D never see an instruction fetch.

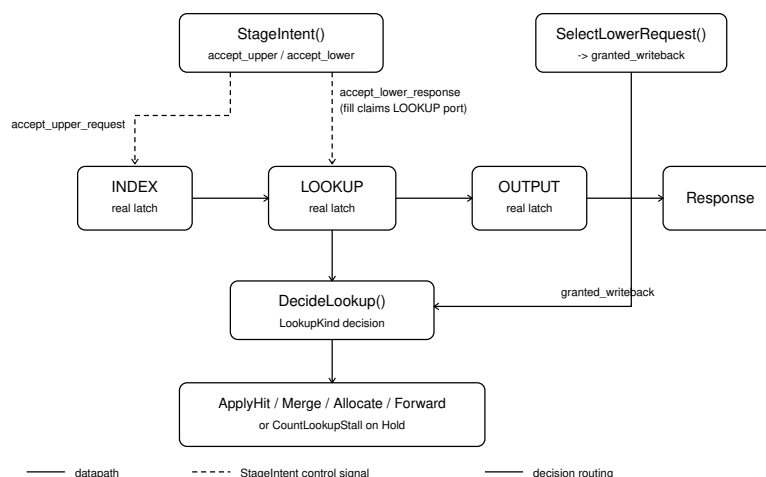
Working

Only PC and IF/ID are real per-cycle registers in this datapath; between Decode/Execute and Execute/Memory there is no shared latch at all, only the direct function call and direct poll described above. The completion queue and pending-data table are real, **Machine**-owned structures distinct from a single-cycle register, which is precisely what lets completion happen out of order across the seven pipelines while retirement is still forced back into strict program order at Writeback.

Cache Architecture

Implementation

Each controller (L1I, L2I, L1D, L2D) is a fully independent instance, implemented in `internal.h` and `replacement.cc`:



- Address mapping.** Each controller decomposes an incoming address using its *own* configured `line.size` and `sets`—never a global constant, since L1 and L2 legitimately differ. The decomposition is: `offset = address mod line.size` (byte position within the line); `block = address - offset` (the line's aligned base address); `set = (block / line.size) mod sets` (which set this line maps to); `tag = (block / line.size) / sets` (its unique identity within that set). LOOKUP then performs a parallel tag comparison across every way in the selected set simultaneously, modeled as a plain loop with no artificial per-way timing cost; finding two valid ways with the same tag in one set is treated as a fatal invariant violation rather than a normal miss, since it can only mean earlier bookkeeping went wrong.
- Miss handling.** On a demand miss, the controller first checks whether an MSHR already covers the same block. If one does, the new request simply **merges** into it as a bounded waiter—no second MSHR, no second lower-level request is ever created for the same block. If no MSHR exists yet, a new one is allocated at the lowest free index: a victim way is selected (invalid ways first, then the configured replacement policy), that way is reserved immediately so nothing else can claim it mid-fill, and a single `kRead` for the entire local line is issued downward. If the evicted victim was valid and dirty, its bytes are captured into a writeback entry before the new line can occupy that way. `DecideLookup` evaluates every possible outcome in one fixed priority order—hit, merge, allocate, writeback-forward,

or hold—and when it must hold, reports the specific reason in a strict precedence: output full, then MSHR waiter full, then MSHR full, then reserved-way conflict, then writeback ordering, then writeback full.

- **Control logic.** On fill, the complete line is installed into its already-reserved way, and every waiting request on that MSHR is replayed in strict **arrival order** at byte granularity—each load’s bytes are captured at its own replay point, each store’s bytes are applied at its own point—so overlapping loads and stores on the same fill are still resolved correctly. `Operate()` is the single mutating control point per cycle, and it runs in a fixed sequence: snapshot the three current intents from one consistent view of state, validate every grant the framework is trying to authorize against those intents (throwing if anything doesn’t match), apply the response and lower-response grants, run the one LOOKUP decision this cycle allows (dispatching to `ApplyHit`, `ApplyMshrMerge`, `ApplyMshrAllocation`, `ApplyWritebackForward`, or just counting the stall on hold), move INDEX into LOOKUP if it vacated, admit one new upper request into INDEX, and finally transition the granted lower-request owner’s state—all from the same snapshot, so nothing is decided on stale information.
- **Response arbitration.** At most one response can travel upward per cycle, so when several are ready simultaneously, `SelectResponse()` picks the single minimum by (`arrival_order`, `response_id`) across three distinct sources: the plain OUTPUT hit response; the oldest ready-and-unsent waiter belonging to each currently filled MSHR; and the oldest unsent acknowledgement waiter belonging to each writeback that has reached `ACK_READY`. This is a genuine two-level minimum—first the best candidate is found within each MSHR and each writeback individually, then those winners are compared against OUTPUT and each other. `ResponseIntent()` is a thin wrapper that just extracts the winning response as a `std::optional`, so nothing downstream needs to know the arbitration happened.
- **Backpressure and stage admission.** `OutputWillDrain` is deliberately conservative: it is only true when OUTPUT currently holds something valid, a response was actually granted this cycle, *and* arbitration specifically chose that OUTPUT candidate over any MSHR or writeback waiter—so OUTPUT is never assumed free just because some response went out. `StageIntent()` computes two admission signals from this: **accept_lower_response**, true only when the incoming response genuinely matches an MSHR fill or writeback ack that is currently `IN_FLIGHT`; and **accept_upper_request**, true only when INDEX is empty right now or is about to vacate into LOOKUP this very cycle.
- **Writebacks.** Every dirty eviction produces a writeback entry that moves through a strict lifecycle: `UNSENT`, then `IN_FLIGHT` once sent downward, then `ACK_READY` once acknowledged, then freed once every waiter on it has been notified. Two `UNSENT` writebacks whose byte ranges overlap are not kept separate—they **merge**, with the older entry surviving and the incoming (newer) bytes winning wherever the ranges overlap, since those are the most up-to-date values. A demand read for that same range is arbitrated fairly against writebacks: it normally wins outright, unless the writeback buffer has genuinely run out of free slots, or the single oldest eligible writeback has already been bypassed by demand grants eight times in a row, at which point that writeback is forced through to guarantee forward progress.
- **Replacement.** **Tree-PLRU** keeps one bit per internal node of a binary tree over the ways; every hit or insertion updates the root-to-leaf path so it points away from the just-accessed way, and victim selection simply walks that same path to the opposite side, falling back to the lowest-numbered eligible way on any tie. **SRRIP** instead keeps a 2-bit re-reference value per way: reset and cold ways start at 3 (furthest from reuse), a hit resets a way straight to 0 (immediate reuse expected), and a fresh insertion is set to 2. Victim selection scans for any way already at 3; if none exists yet, every way’s counter is incremented by one (saturating at 3) and the scan repeats, which is what lets a line age into eviction eligibility over time rather than being picked arbitrarily.

Working

Each controller runs a three-stage pipeline every cycle: **INDEX** → **LOOKUP** → **OUTPUT**. A request entering at cycle N reaches INDEX at $N+1$, LOOKUP evaluates it at $N+2$, and a hit response is available at $N+3$; a blocked stage retains its exact entry without repeating allocations or counter events. `Operate()` ties this together each cycle: it snapshots the current intents, validates the framework’s grants against them, then applies exactly one LOOKUP decision, one INDEX admission, and one lower-request transition—so a miss (MSHR allocation) and a hit (immediate response) are handled by the same three-stage machinery, differing only in which `Apply*` function fires. The instruction-side (L1I/L2I) and data-side (L1D/L2D) hierarchies run this identical logic completely independently, with no shared state between them.

Per-cycle bandwidth is bounded at one each: upper request/lower response accepted, upper/lower response/request enqueued, one LOOKUP. Unused bandwidth does not carry over.

Testing and Verification

The starter provides seven integration tests via `tests/run_tests.sh` (cycle/state match, evictions, unequal-config, hazard stalls, `invalid_config`, `multi-cycle-config`, `reset-reproducibility`); all seven tests passed.